

INFO 6205 Team Project Report

Lyuxiong Zhang 002300481

Chengya Xu 002820124

Shuhang Sun 002087017

Monte Carlo Tree Search (MCTS) is a heuristic search algorithm commonly used for decision-making in complex games. In this report, we implemented MCTS for two strategic games: Tic-tac-toe and Connect Four. Our implementation demonstrates the algorithm's ability to effectively navigate game spaces and achieve optimal or near-optimal performance. The results show significant improvements in computational efficiency and game-winning strategies through various optimizations and heuristics.

Our implementation involves four essential phases: selection, expansion, simulation, and backpropagation. The selection phase ensures optimal balance between exploration and exploitation of the game tree. Expansion involves adding new nodes to the tree when encountering previously unexplored game states. The simulation phase executes randomized or heuristic-based playouts to predict potential game outcomes. Finally, the backpropagation step updates the nodes along the simulation path based on results, refining the algorithm's future decisions. Key sections of the implementation are documented through relevant code snippets.

We introduced several heuristics and optimization strategies to enhance performance, including refined node selection and simulation techniques. Comparisons of algorithm performance before and after these improvements demonstrate considerable efficiency gains. Metrics clearly show increased accuracy and reduced computational time, highlighting the impact of targeted optimizations.

1. Tic-tac-toe

a. Rules

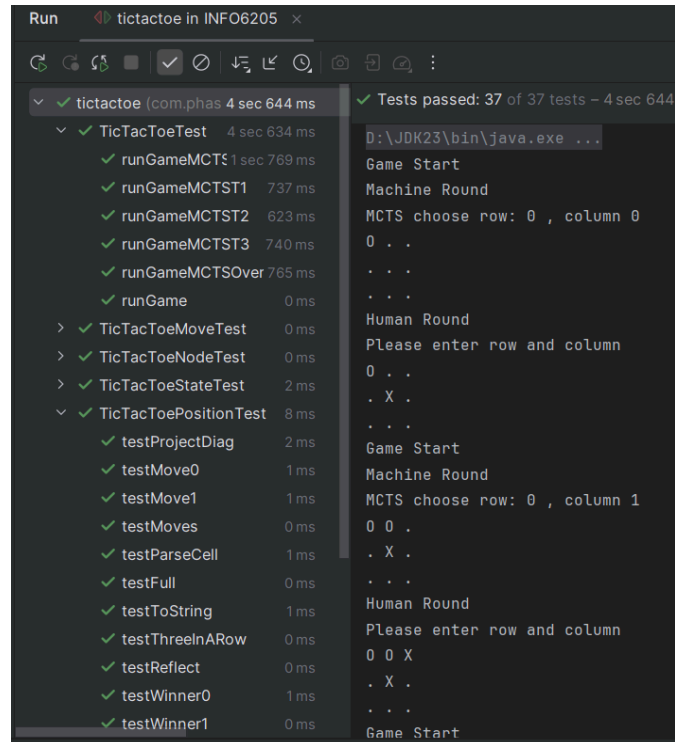
The game is played on a 3x3 square board. Two players take turns placing their markers in the space, and one player uses "X", another player uses "O". The goal of the players is to form a straight line of three identical markers horizontally or diagonally, and the first player to successfully form a line wins. If the board is filled but no players form a line, the game ends in a tie. Players can only place one marker in one space per turn, and once placed they cannot move or change the game. Usually, the player "X" goes first, and then the two players alternate until the game is won or tied.

b. How it works

When the machine kicks off the game, the player enters two numbers to select the position, as the first number indicates the columns, and the second figure represents the rows (which are both started from zero).

After the grid fills out with “O” and “X”, there are three possibilities of the outcome. 1) The machine wins the game 2) The player wins the game 3) Both sides draw.

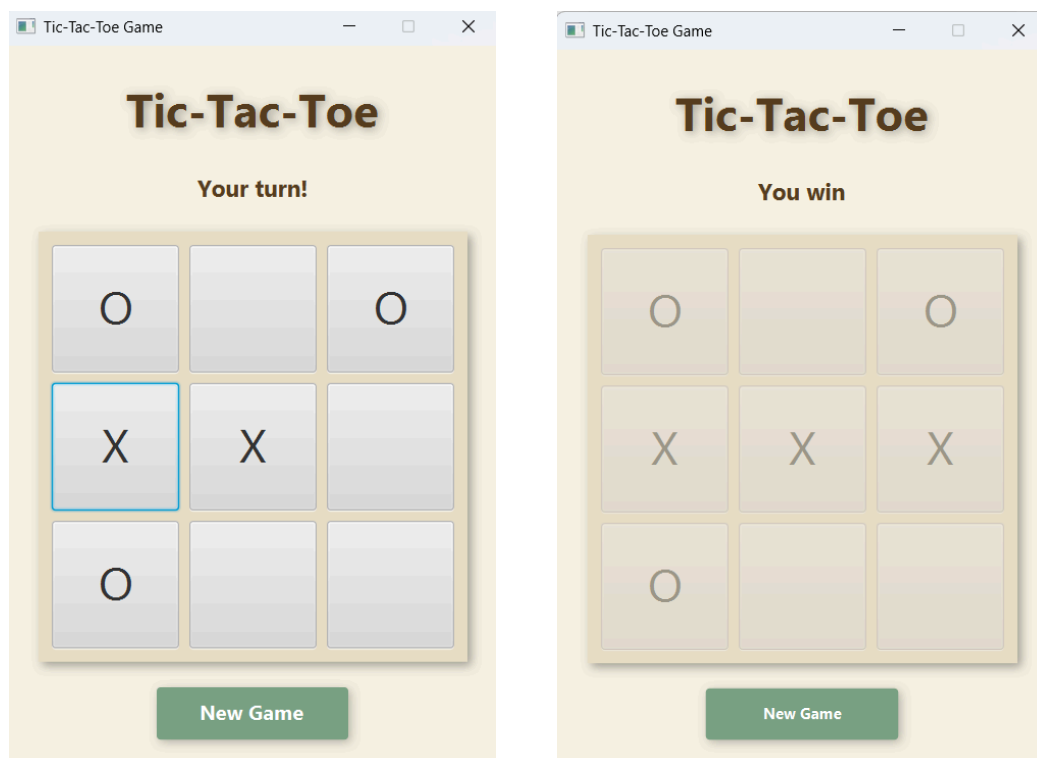
c. Unit Test



```
Run tictactoe in INFO6205 x
✓ tictactoe [com.phas 4 sec 644 ms]
  ✓ TicTacToeTest 4 sec 634 ms
    ✓ runGameMCTS 1 sec 769 ms
    ✓ runGameMCTST1 737 ms
    ✓ runGameMCTST2 623 ms
    ✓ runGameMCTST3 740 ms
    ✓ runGameMCTSOVer 765 ms
    ✓ runGame 0 ms
  > ✓ TicTacToeMoveTest 0 ms
  > ✓ TicTacToeNodeTest 0 ms
  > ✓ TicTacToeStateTest 2 ms
  > ✓ TicTacToePositionTest 8 ms
    ✓ testProjectDiag 2 ms
    ✓ testMove0 1 ms
    ✓ testMove1 1 ms
    ✓ testMoves 0 ms
    ✓ testParseCell 1 ms
    ✓ testFull 0 ms
    ✓ testToString 1 ms
    ✓ testThreeInARow 0 ms
    ✓ testReflect 0 ms
    ✓ testWinner0 1 ms
    ✓ testWinner1 0 ms
  ✓ Tests passed: 37 of 37 tests - 4 sec 644
  D:\JDK23\bin\java.exe ...
  Game Start
  Machine Round
  MCTS choose row: 0 , column 0
  0 . .
  . . .
  . . .
  Human Round
  Please enter row and column
  0 . .
  . X .
  . . .
  Game Start
  Machine Round
  MCTS choose row: 0 , column 1
  0 0 .
  . X .
  . . .
  Human Round
  Please enter row and column
  0 0 X
  . X .
  . . .
  Game Start
```

Node: Unit tests are in src/test/java/com/phasmidsoftware/dsaipg/projects/mcts

d. Sample Run



2. Connect Four

a. Rules

The game uses a vertical board with six rows and seven columns, and two players take turns putting pieces of their own color at the top of the board (usually red for one player and yellow for the other). Due to gravity, the piece will fall into the empty space at the bottom of the column. The goal of the player is to form four pieces of their own color first in horizontal, vertical, or diagonal directions. The first player to make a successful four wins. If the board is filled but no player fads, the game ends in a draw. The game is usually played by one of the players first, and then the two players alternate until the game is won or tied.

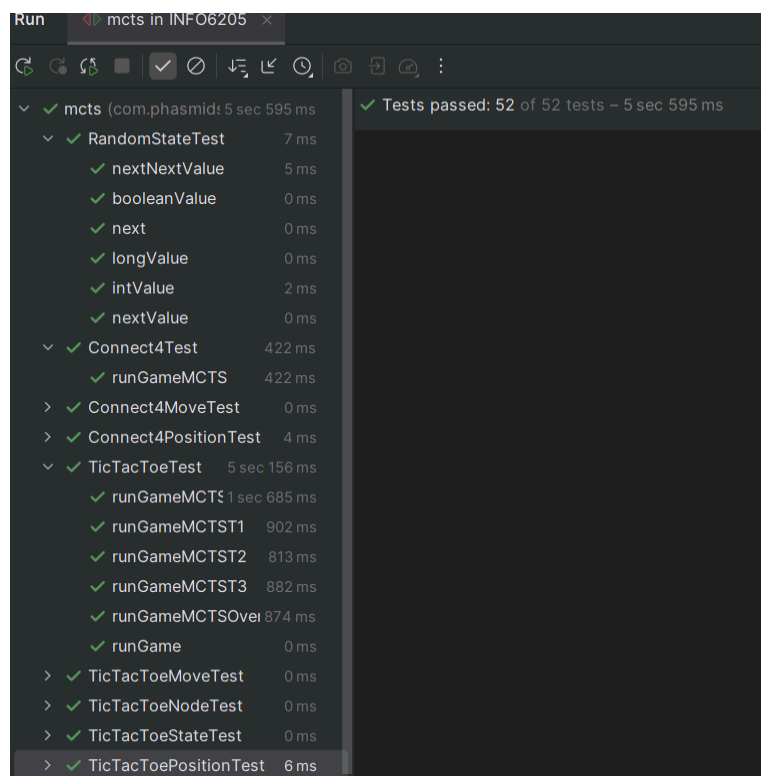
Unlike tic-tac-toe, four-piece is more strategic because pieces must obey the rule of gravity and can only be placed from above and stacked on top of existing pieces.

b. How it works

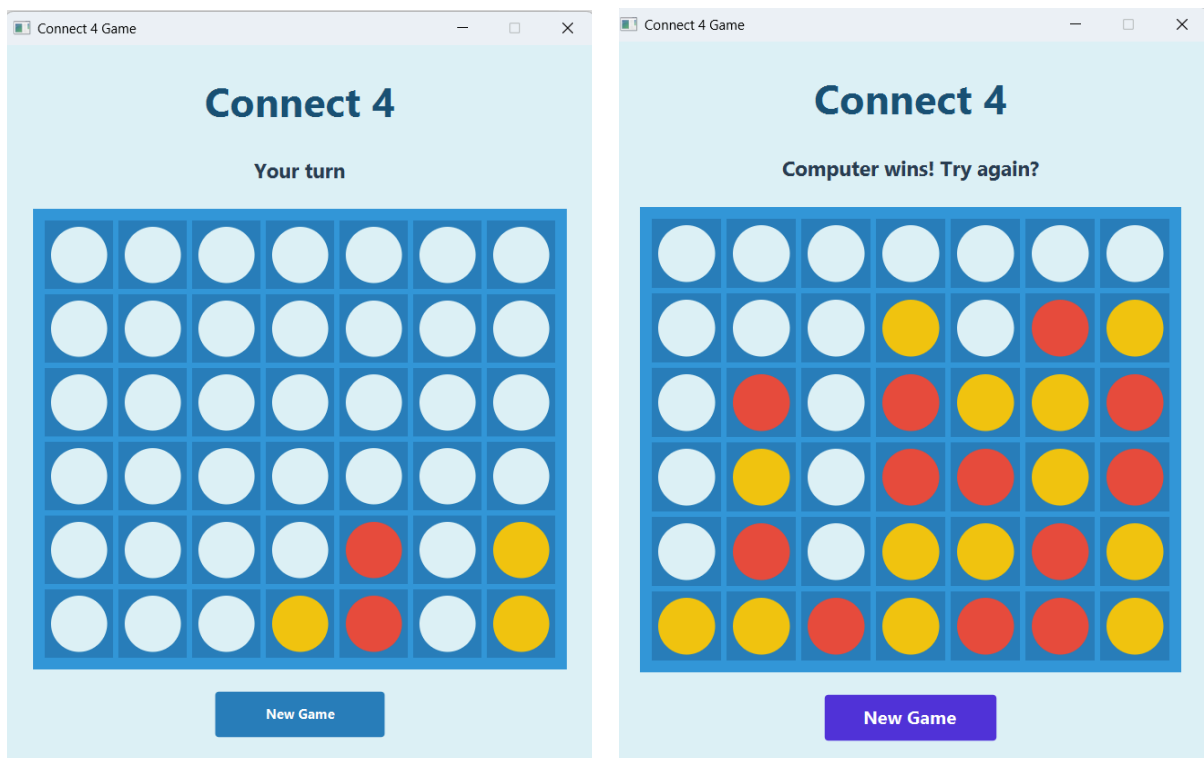
When the game begins, the player selects a column by entering a number (starting from zero). The machine then places the player's piece in the lowest available row within that column.

As the grid fills with "O" and "X" pieces, the game can end in one of three outcomes: 1)The machine wins the game. 2)The player wins the game. 3)The game ends in a draw.

c. Unit Test



d. Sample Run



Running Guide

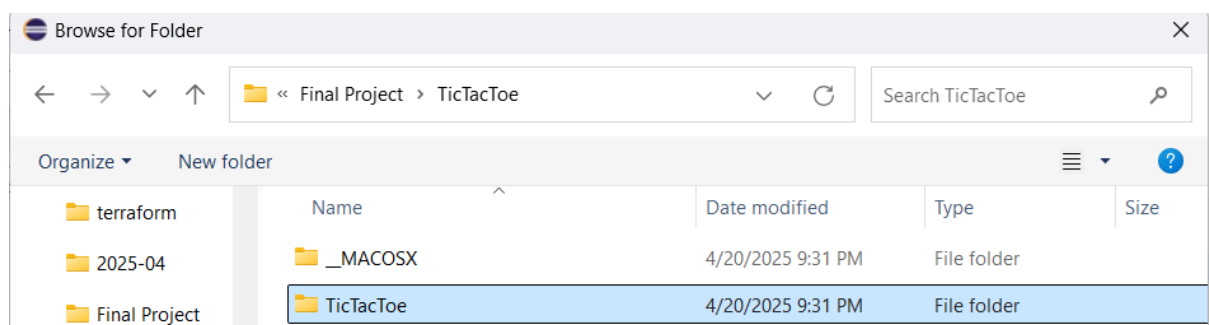
Download TicTacToe and Connect Four from the following Repository

<https://github.com/SetsunaWithExia/DSAIPG.git>

<https://github.com/Sunshuhang924/DSAIPG.git>

<https://github.com/Stella051222/DSAIPG.git>

Open Eclipse Import file project



Set Configuration VM arguments

VM arguments:

```
--module-path D:\JavaFXSDK\openjfx-21.0.5_windows-x64_bin-sdk\javafx-sdk-21.0.5\lib --add-modules=javafx.controls,javafx.fxml
```

Set Difficulty of the Game

MainController.java

```
private static final int EASY = 20;  
private static final int MEDIUM = 500;  
private static final int HARD = 100000;  
private static final int DIFFICULTY = EASY; //Iteration times of MCTS
```

Run Game

Run TicTacToe/src/main.java

Optimization

Upper Confidence Bound Bound applied to Trees (UCT)

We referenced the following function for optimization from Monte Carlo Tree Search Guidance¹:

$$UCT(vi, v) = \frac{Q(vi)}{N(vi)} + c \sqrt{\frac{\ln N(v)}{N(vi)}}$$

Specially $c = 1.42$

```
double uct = (double)child.val() / child.vis() +  
    C * Math.sqrt(Math.log(vis)/child.vis());
```

Performance

TicTacToe	Human First	Machine First	Human First	Machine First	Human First	Machine First
MCTS Iterate Times	362880	362880	1000	1000	20	20
Machine Win	4	8	0	8	0	2
Human Win	0	0	0	0	5	0

¹ <https://int8.io/monte-carlo-tree-search-beginners-guide/>

Draw	61	2	10	2	0	4
Machine Win Rate	6.15%	80%	0%	80%	0%	0%
Draw Rate	93.85%	20%	100%	20%	0%	50%
Human Win Rate	0%	0%	0%	0%	100%	50%

Connect4	Human First	Machine First
MCTS Iterate times	1000	1000
Machine Win	5	9
Human Win	0	1
Draw	1	0
Machine Win Rate	83.33%	90%
Draw Rate	16.67%	0%
Human Win Rate	0.00%	10%

Note: All tests were performed by authors and try to win the game.

Reflection and Challenges

The project presented several technical challenges, notably in algorithm optimization and handling complex game logic. We documented strategies to overcome these challenges, emphasizing the learning curve involved in debugging intricate logic, optimizing algorithm efficiency, and enhancing team collaboration during software development.

Future Work and Improvements

Potential improvements include search data persistence, optimizing the UCT function for better performance, adding parallel processing capabilities for enhanced efficiency, and adapting the MCTS implementation to more complex games. Future work will further test the robustness, adaptability, and applicability of our Monte Carlo Tree Search algorithm.